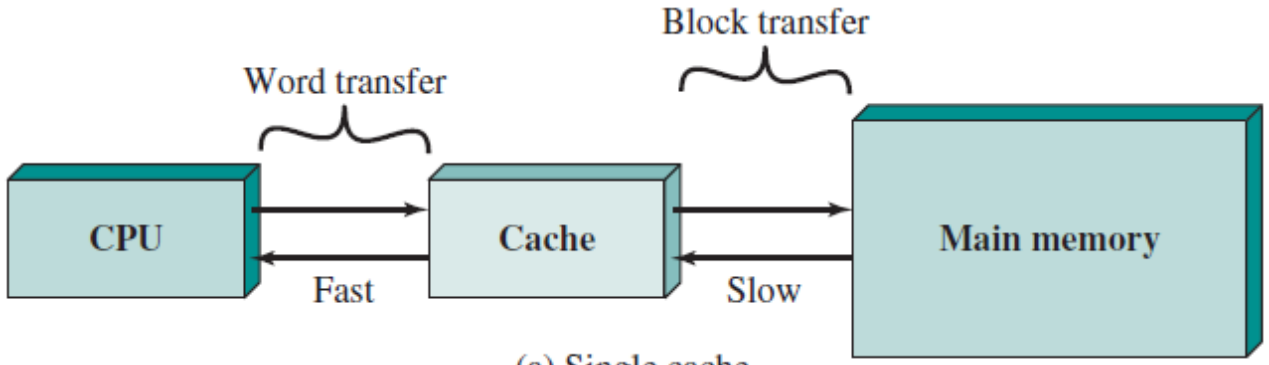
Memory Organization: Cache Memory

Shobhanjana Kalita,
Dept. of CSE, Tezpur University

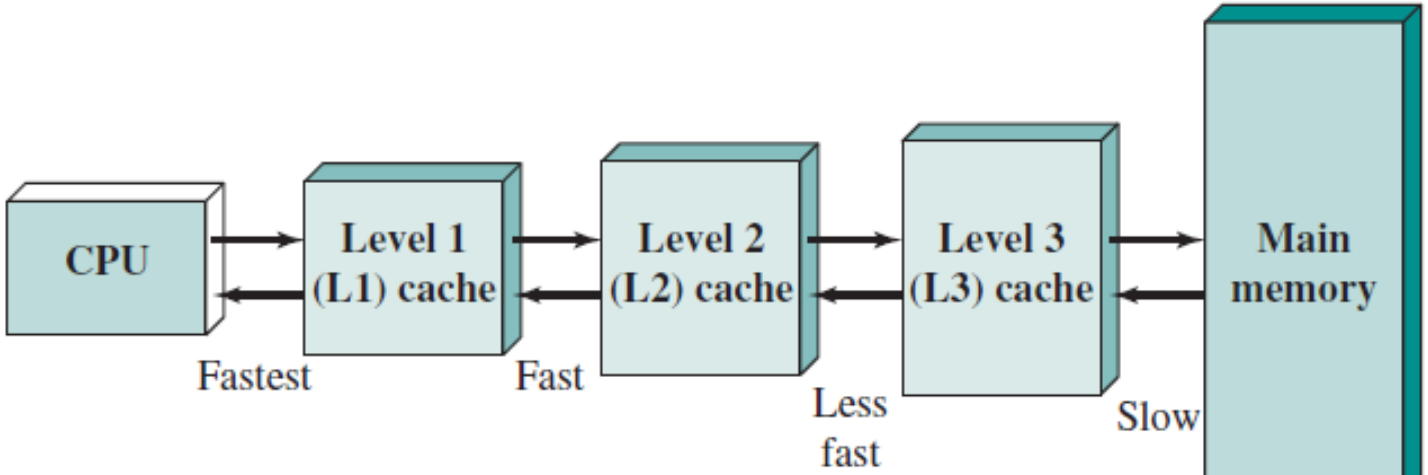
Slides courtesy: Computer Architecture and Organization, 9th Ed, W. Stallings

Cache Memory

- Cache memory is *designed to combine* the memory access time of expensive, highspeed memory with the large memory size of less expensive, lower-speed memory
  - A relatively large, slow main memory is combined together with a smaller, faster cache memory.



(a) Single cache



(b) Three-level cache organization

Cache Memory

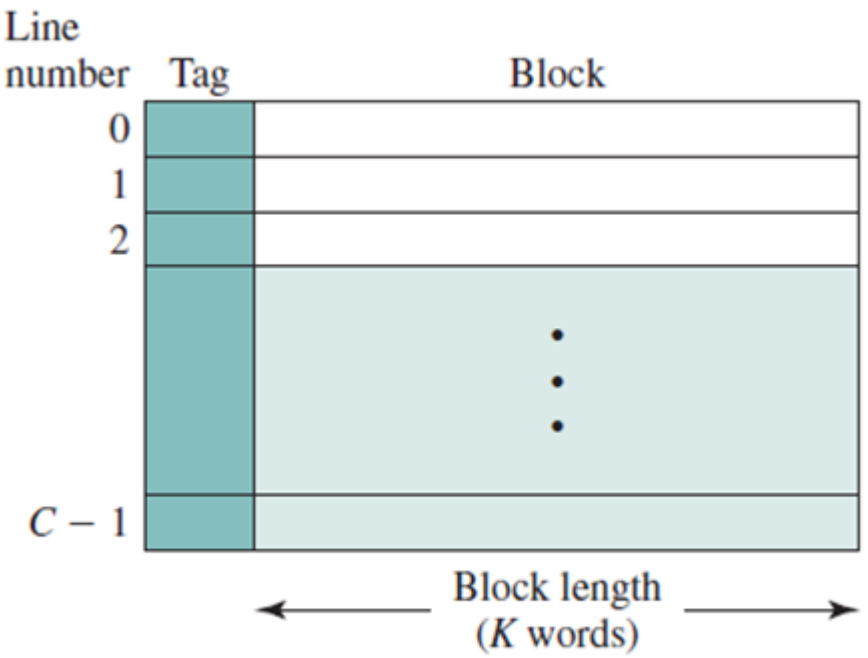
- Cache contains a copy of portions of main memory
  - When the processor attempts to read a word of memory, a check is made to determine if the word is in the cache
  - ***If yes***, the word is delivered to the processor
  - ***If no***, a block of main memory containing the word, and consisting of some fixed number of words, is read into the cache and then the word is delivered to the processor
- Based on the phenomenon of **locality of reference**
  - ***when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block***

Cache Memory

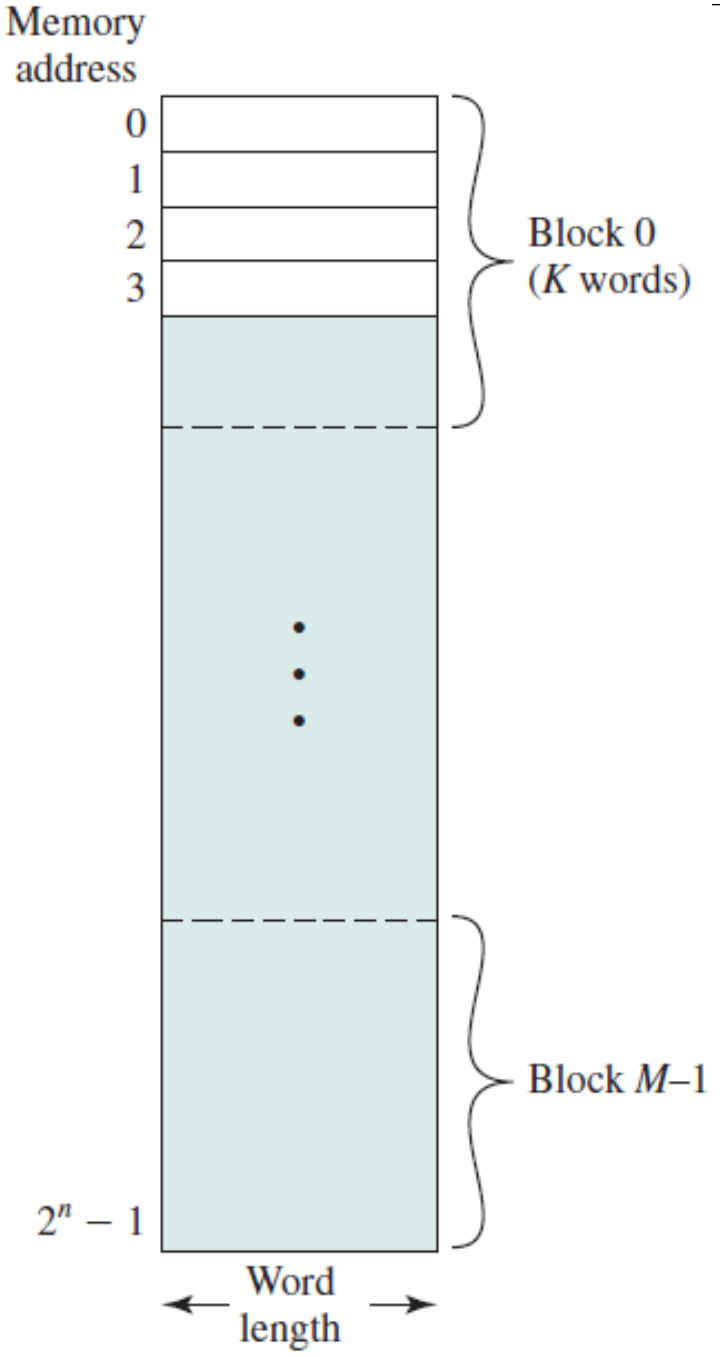
- **Locality of Reference**
  - Most of the execution time of a program is spent on loops, and procedures (functions) that call one another
  - A small set of instruction is executed repeatedly, while the rest of the instructions are executed less frequently
- Locality of reference manifests in two ways:
  - **Temporal:** A recently executed instruction is likely to be executed again very soon
  - **Spatial:** Instructions that are near (in terms of sequencing) to an instruction recently executed are likely to be executed soon
- When such highly active portions of program are placed in a faster cache memory, execution time is reduced significantly

Cache Memory

Main memory is further organized into **blocks** of K words
Cache is organized into **lines** such that one line can hold one block (or K words)



(a) Cache



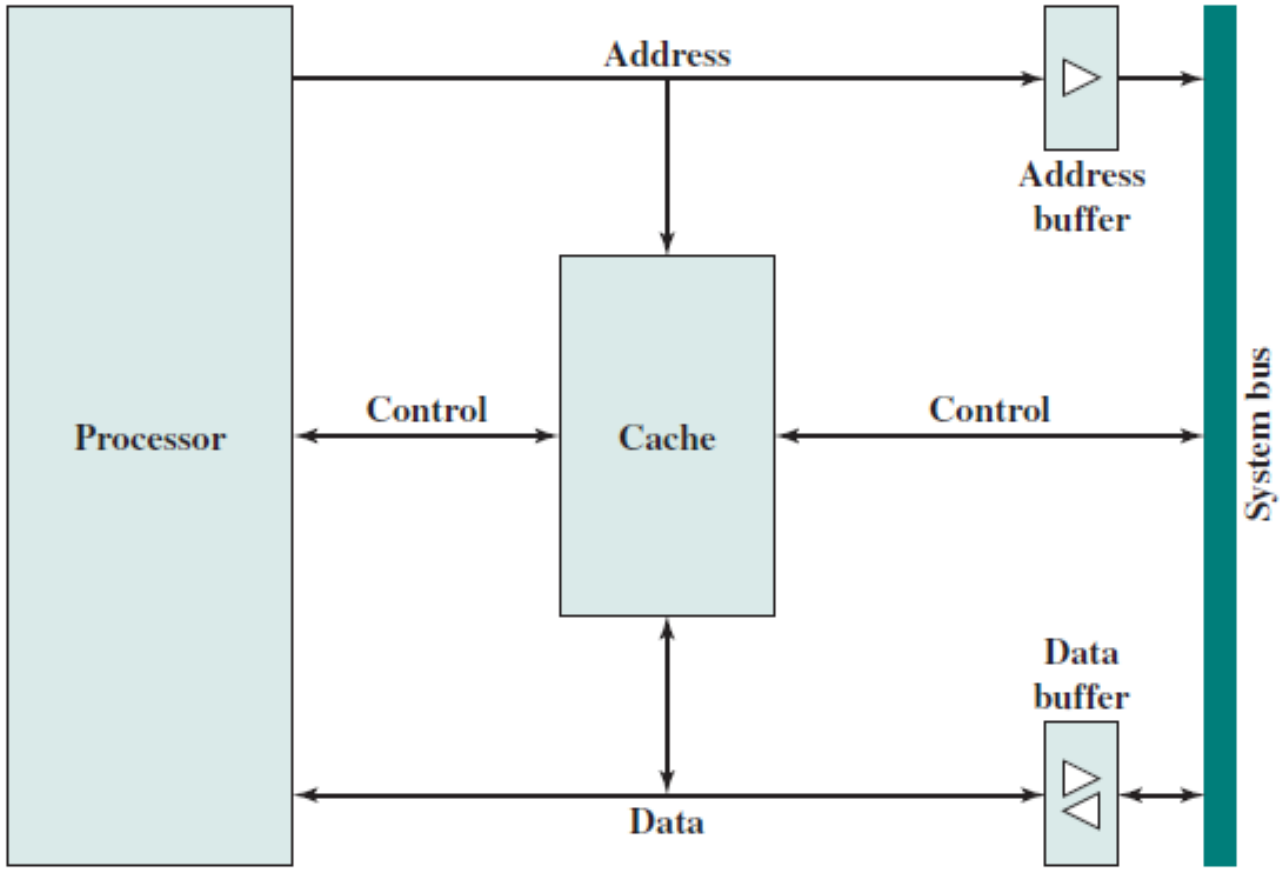
(b) Main memory

Cache Memory

- Main memory consists of up to 2^n addressable words, with each word having a unique n-bit address.
- For mapping purposes, grouped into a number of fixed-length blocks of K words each.
  - $M = (2^n)/K$ blocks in main memory.
  - The cache consists of m lines each of which that can hold one block of data
  - Each line contains K words + a tag of a few bits
  - Also includes control bits (e.g. a bit to indicate whether the line has been modified since being loaded into the cache)
  - Length of a line, not including tag and control bits, is **line size**
  - **Tag** that identifies which particular block is currently being stored ; usually a portion of the main memory address
- No. of lines, **m << M**, the main memory size

Cache Memory

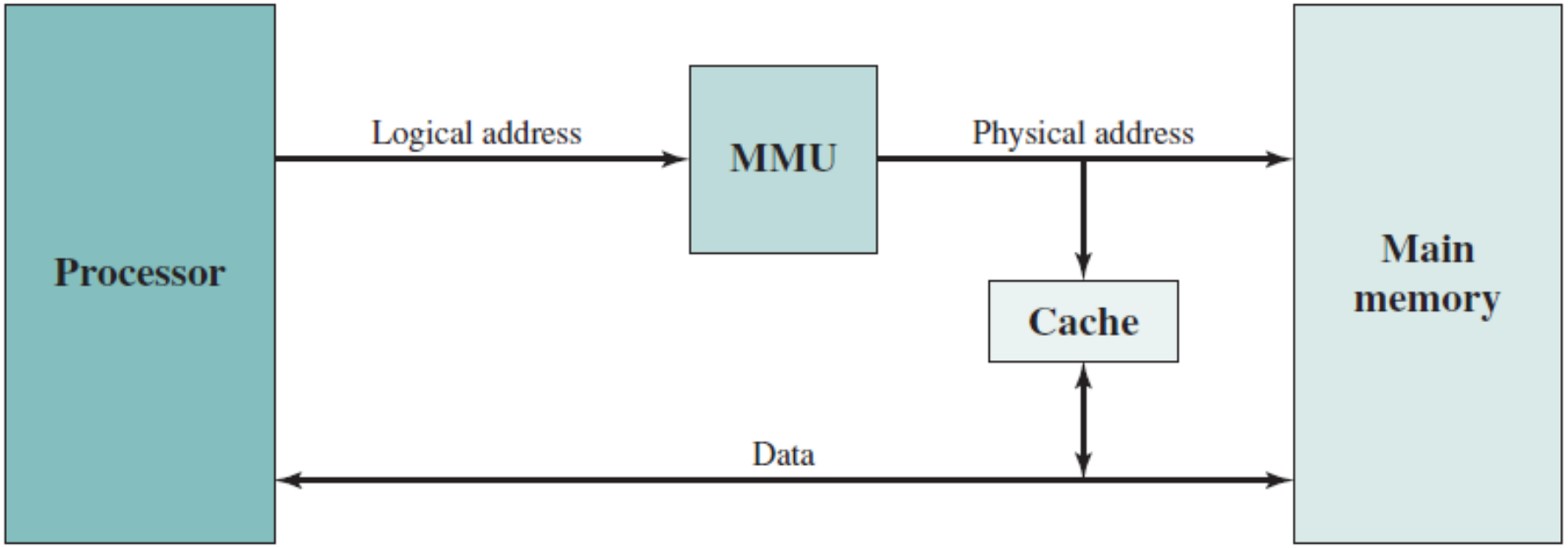
- Cache connects to the processor via data, control, and address lines.
- Data and address lines also attach to data and address buffers, which attach to a system bus to the main memory.
- Processor need not know about the existence of the cache. It simply issues Read and Write requests using addresses that refer to memory locations.
- The cache control circuitry determines whether the requested address/word currently exists in the cache.



Typical Cache Organization

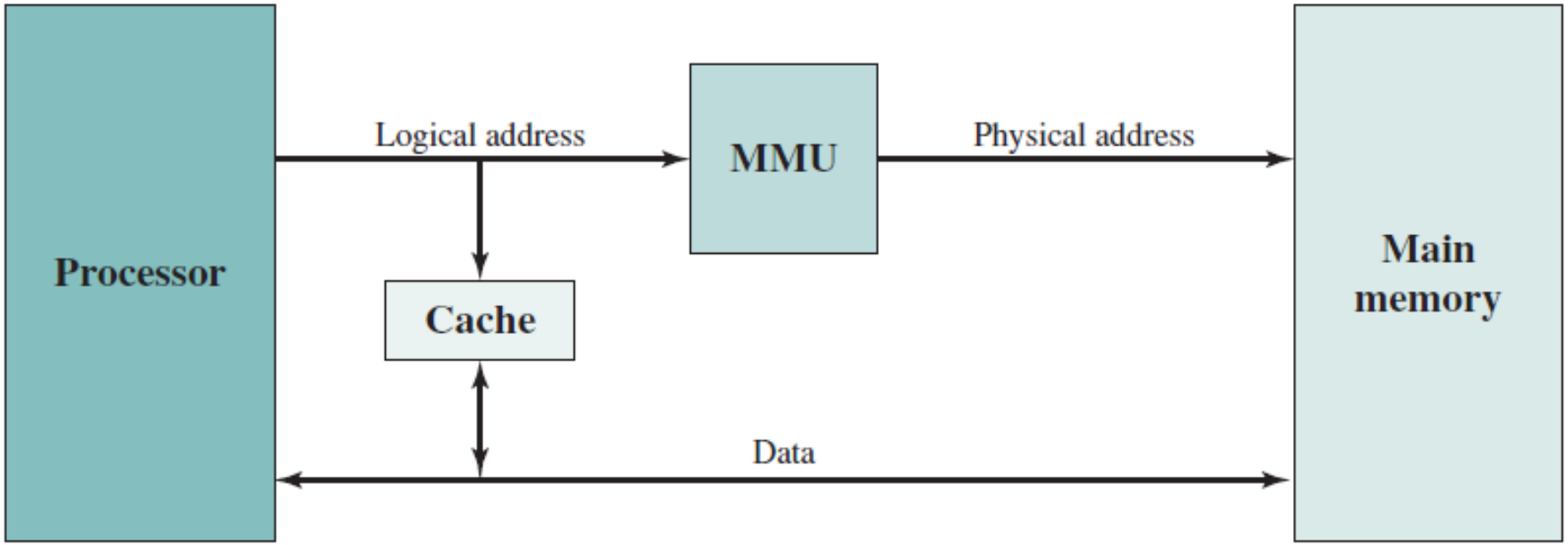
Cache Memory - Addresses

- A **physical cache** stores data using main memory physical addresses



Cache Memory – Addresses

- A **logical cache** (or virtual cache) stores data using virtual addresses (not actual)
  - hardware memory management unit (MMU) translate the virtual address into a physical address in main memory



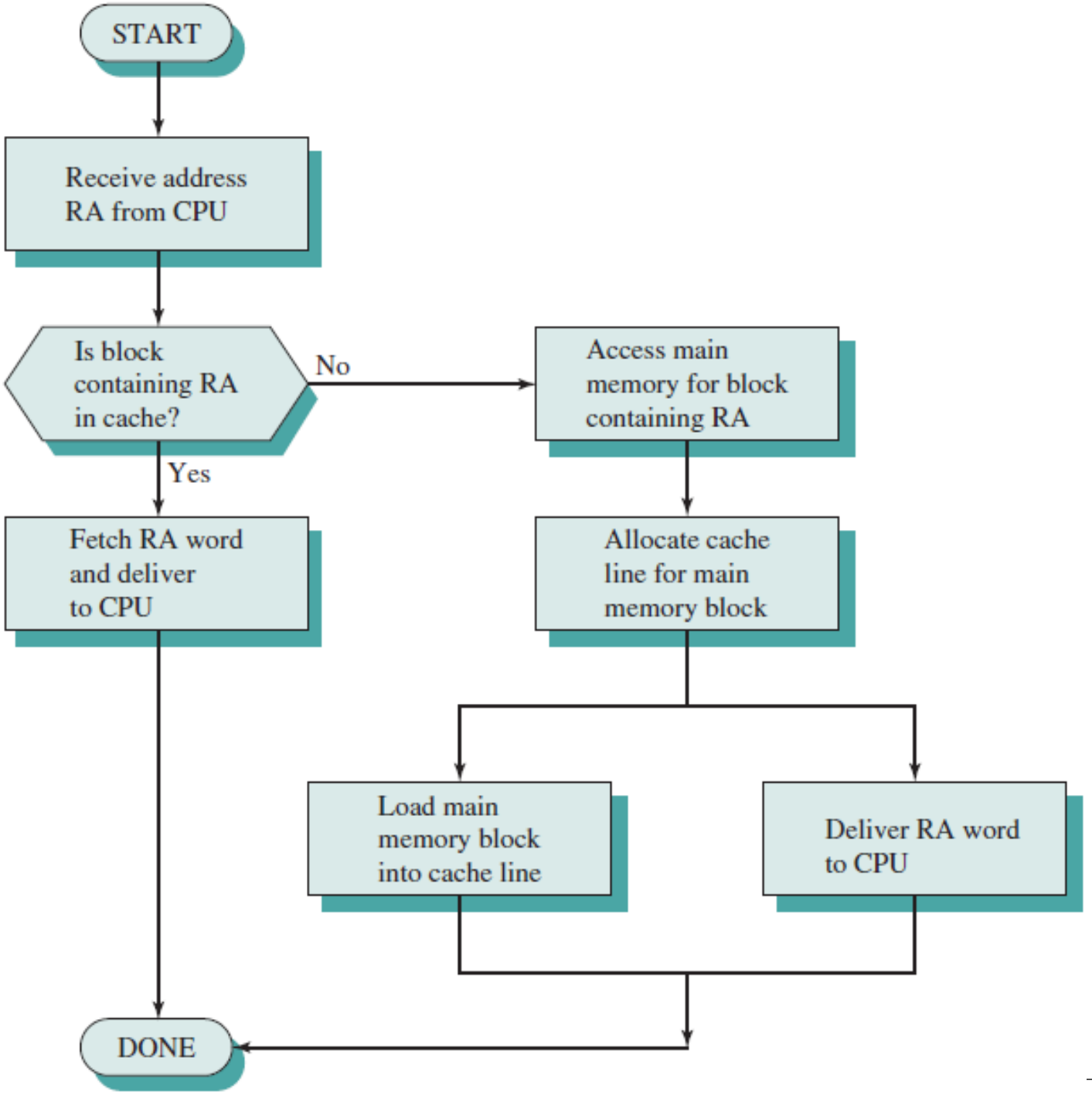
Cache Hit

- When the processor requests for an address, and the address is in the cache, then it is said to be a **cache hit**.
- In case of a **read** operation, the requested word is read from the cache; memory is not involved.
- In case of a **write** operation for a word that is in the cache, then there are two *write policies* to handle it –
  - **Write through** - Immediately update in the main memory, all changes that have been made in the cache.
    - Main memory data is always valid for any other modules (say I/O) that wishes to access it
    - Simpler but creates substantial memory traffic
  - **Write back (or copy back)** – Updates are made only in the cache. Set a *dirty* bit (or *use* or *modified* bit) against the cache line to indicate it has been updated. It is updated back in memory only when the cache line is to be replaced.
    - Minimizes memory writes
    - Portions of the main memory may be invalid, and accesses for all modules to be done via cache.
    - Complex circuitry and potential bottleneck

Cache Miss

- When the processor requests for an address, and the address is not found in the cache, then it is said to be a **cache miss**.
- In case of **read** operation, there are two ways of handling it –
  - the entire block of words containing the requested word is copied from MM into the cache. After that, the requested word is forwarded to the processor.
  - or the entire block is copied from MM to cache, and at the same time the request word is forwarded to processor. This is called **load through** or **early restart** – requires complex circuitry but somewhat fast.
- In case of **write** operation, it is handled according to the write policy –
  - in write through policy, the addressed word is written into the MM and then the block containing the word is copied to the cache
  - In write back policy, the block containing the addressed word is first brought into the cache, and then word in the cache is overwritten with the new information.

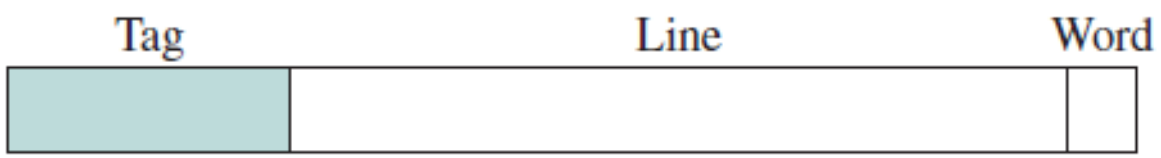
Flowchart for read request by the processor, that uses load through for cache miss



Cache Memory - Mapping

- Since there are fewer cache lines than main memory blocks, a method for **mapping** blocks to cache lines is needed
  - i.e. To determine which block goes to which line
- Based on the type of **mapping function**, the cache organization may differ
- 3 techniques
  - Direct
  - Associative
  - Set Associative

Cache – Direct Mapping



- Maps each block of main memory into only one possible cache line. The mapping is expressed as
  - $i = j \text{ modulo } m$, where
  - i = *cache line number*, j = *main memory block number*, m = *no. of lines in the cache*
- Thus first *m* blocks map to the *m* cache lines in a one-to-one manner; similarly next *m* blocks map to the *m* cache lines …
- Mapping function is easily implemented using the main memory address
- Each main memory address can be viewed as consisting of three fields.
  - **w** LSBs identify a unique word or byte within a block of main memory
  - Remaining **s** bits specify which block of memory
  - **s-r** MSBs is the tag and **r** bits to specify cache line

Cache – Direct Mapping

Address length $= (s + w)$ bits

Number of addressable units $= 2^{s+w}$ words or bytes

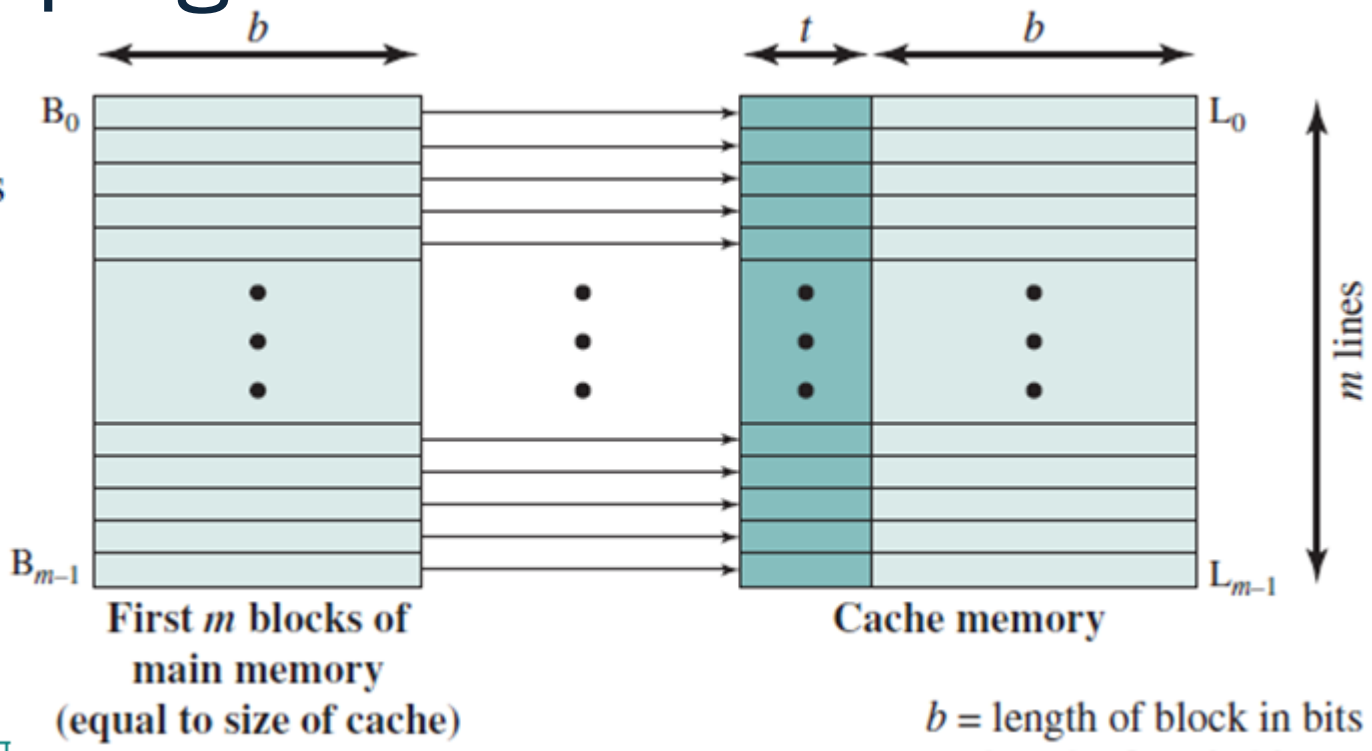
Block size = line size = $2w$ words or bytes

Number of blocks in main memory $= \dfrac{2^{s+w}}{2^w} = 2^s$

Number of lines in cache $= m = 2r$

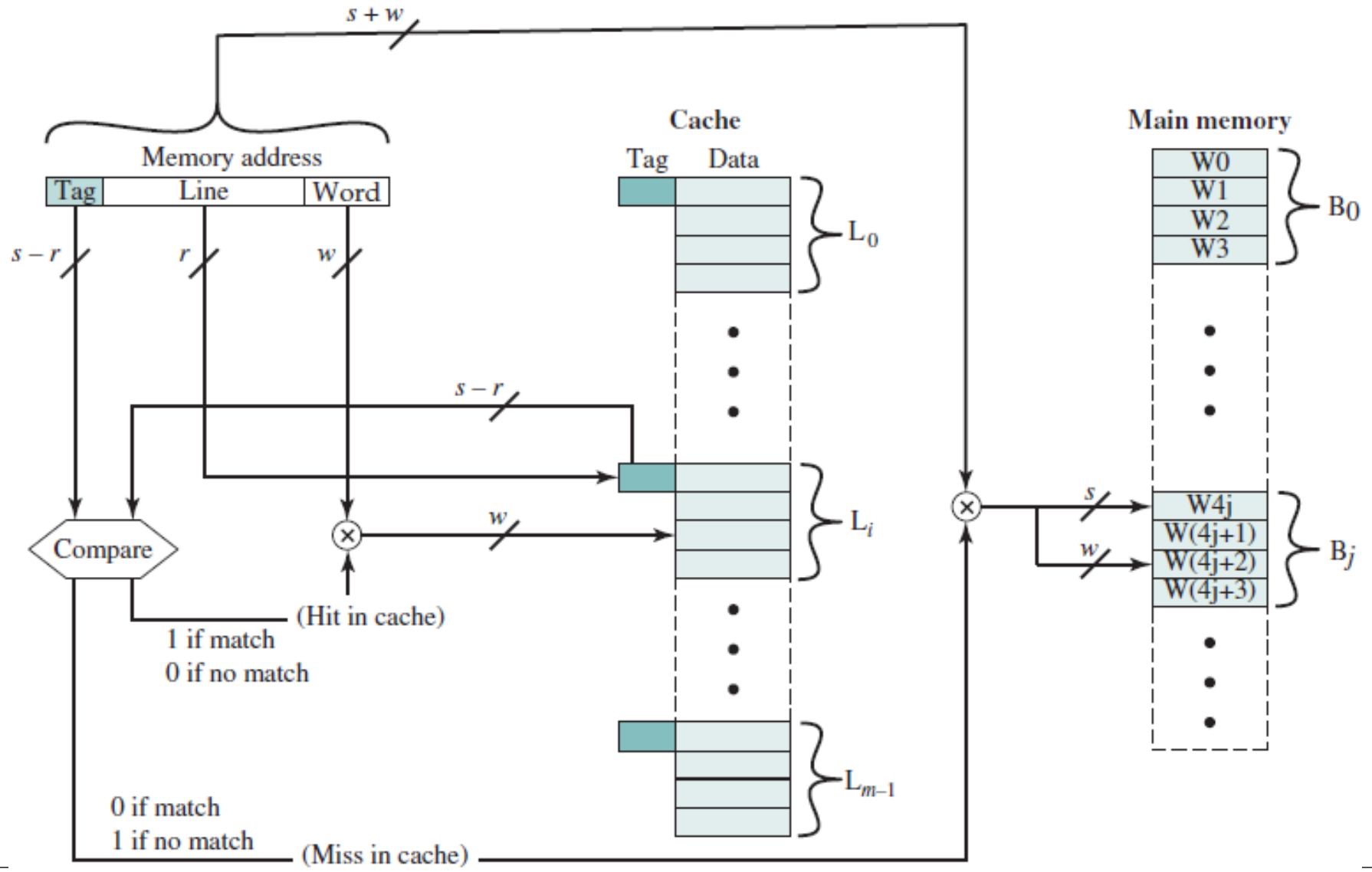
Size of cache $= 2^{r+w}$ words or bytes

Size of tag $= (s - r)$ bits



Cache line	Main memory blocks assigned
0	$0, m, 2m, \ldots, 2^s - m$
1	$1, m + 1, 2m + 1, \ldots, 2^s - m + 1$
⋮	⋮
$m - 1$	$m - 1, 2m - 1, 3m - 1, \ldots, 2^s - 1$

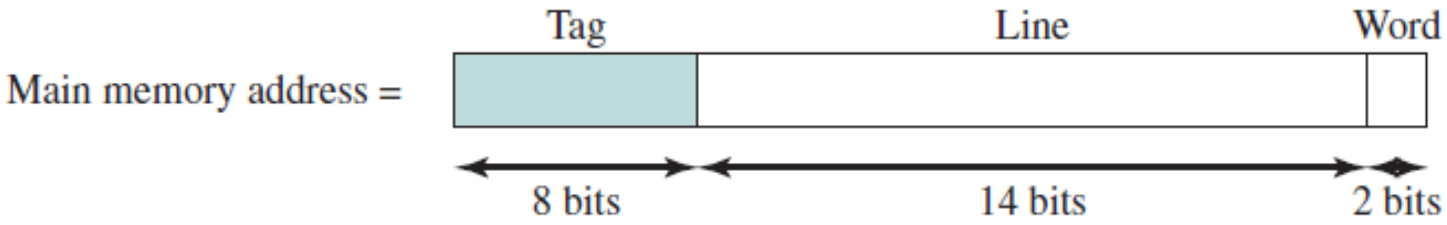
Cache – Direct Mapping



Cache – Direct Mapping

- Example
  - The cache can hold 64 Kbytes.
  - Data are transferred between main memory and the cache in blocks of 4 bytes each. This means that the cache is organized as $16K = 2^{14}$ lines of 4 bytes each.
  - The main memory consists of 16 Mbytes, with each byte directly addressable by a 24-bit address ($2^{24} = 16M$). Thus, for mapping purposes, we can consider main memory to consist of 4M blocks of 4 bytes each.

14-bit line number is used as an index into the cache to access a particular line, **8-bit tag number** matches the tag number currently stored in that line, **2-bit word number** is used to select one of the 4 bytes in that line



Cache Line	Starting Memory Address of Block
0	000000, 010000, …, FF0000
1	000004, 010004, …, FF0004
⋮	⋮
$2^{14} - 1$	00FFFC, 01FFFC, …, FFFFFC

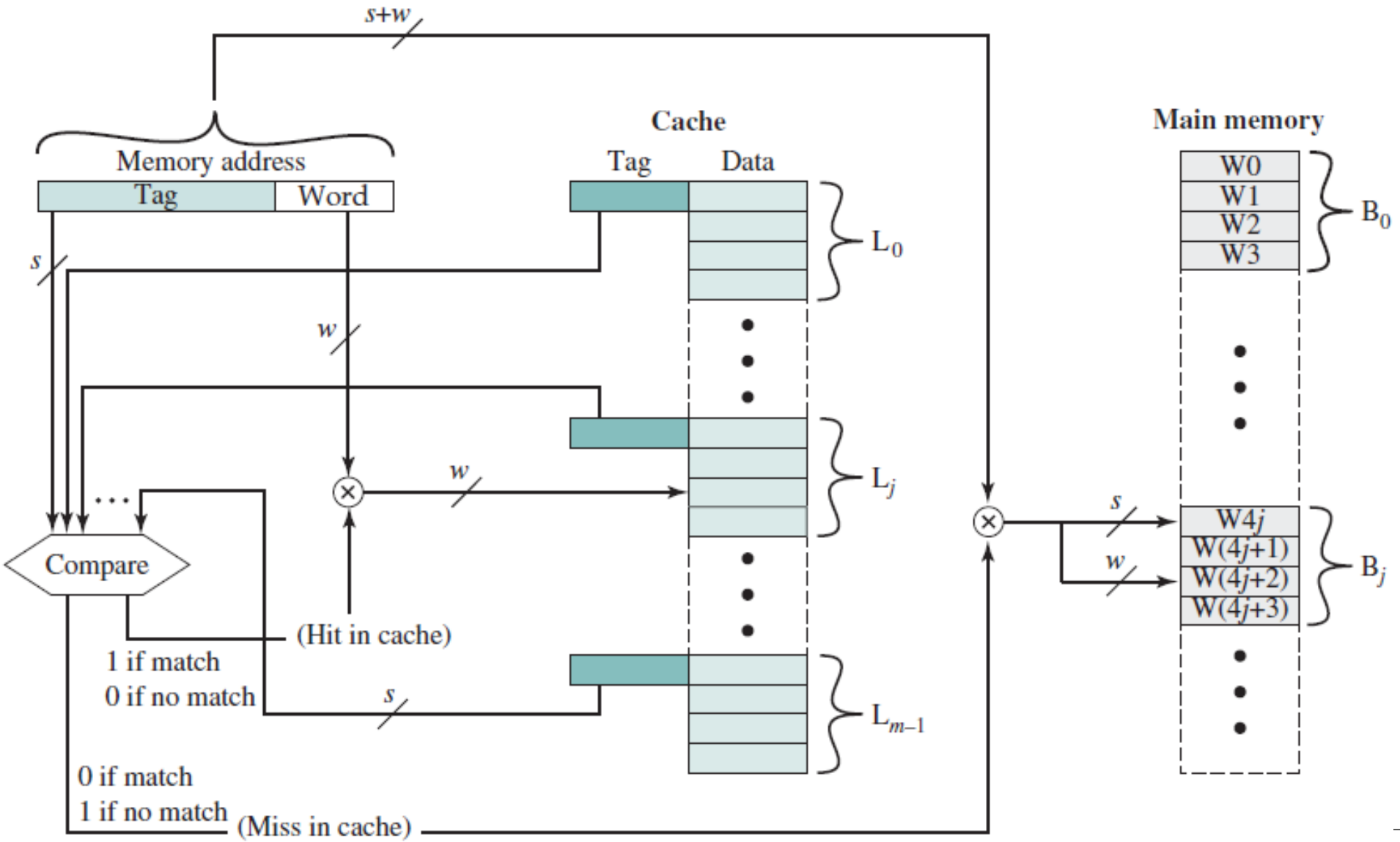
Cache – Direct Mapping

- **Advantage:**
  - The direct mapping technique is simple and inexpensive to implement.
- **Disadvantage:** Since there is a fixed cache location for any given block,
  - reference words repeatedly from two different blocks that map into the same line, then the blocks will be continually swapped in the cache - ***thrashing***
  - hit ratio will be low

Cache - Associative Mapping

- Associative mapping overcomes the disadvantage of direct mapping by ***permitting each main memory block to be loaded into any line of the cache***
- Memory address is interpreted as a Tag and a Word field
  - Tag field uniquely identifies a block of main memory
  - To determine whether a block is in the cache, the cache control logic must simultaneously examine every line's tag to find a block

Cache – Associative Mapping



Cache – Associative Mapping

- No field in the address corresponds to the line number, so that the number of lines in the cache is not determined by the address format

$$\text{Address length} = (s + w) \text{ bits}$$
$$\text{Number of addressable units} = 2^{s+w} \text{ words or bytes}$$
$$\text{Block size} = \text{line size} = 2^{w} \text{ words or bytes}$$
$$\text{Number of blocks in main memory} = \frac{2^{s+w}}{2^{w}} = 2^{s}$$
$$\text{Number of lines in cache} = \text{undetermined}$$
$$\text{Size of tag} = s \text{ bits}$$

- **Advantage:** flexibility as to which block to replace when a new block is read into the cache; chosen using *replacement algorithms* to maximize hit ratio
- **Disadvantage:** complex circuitry required to examine the tags of all cache lines in parallel

Cache – Set Associative Mapping

- Combines the strengths of both the direct and associative approaches while reducing their disadvantages
- Cache consists of a number **sets**, each of which consists of a number of **lines** such that

$$m = v \times k$$

$$i = j \text{ modulo } v$$

  - Where,

i = cache set number

j = main memory block number

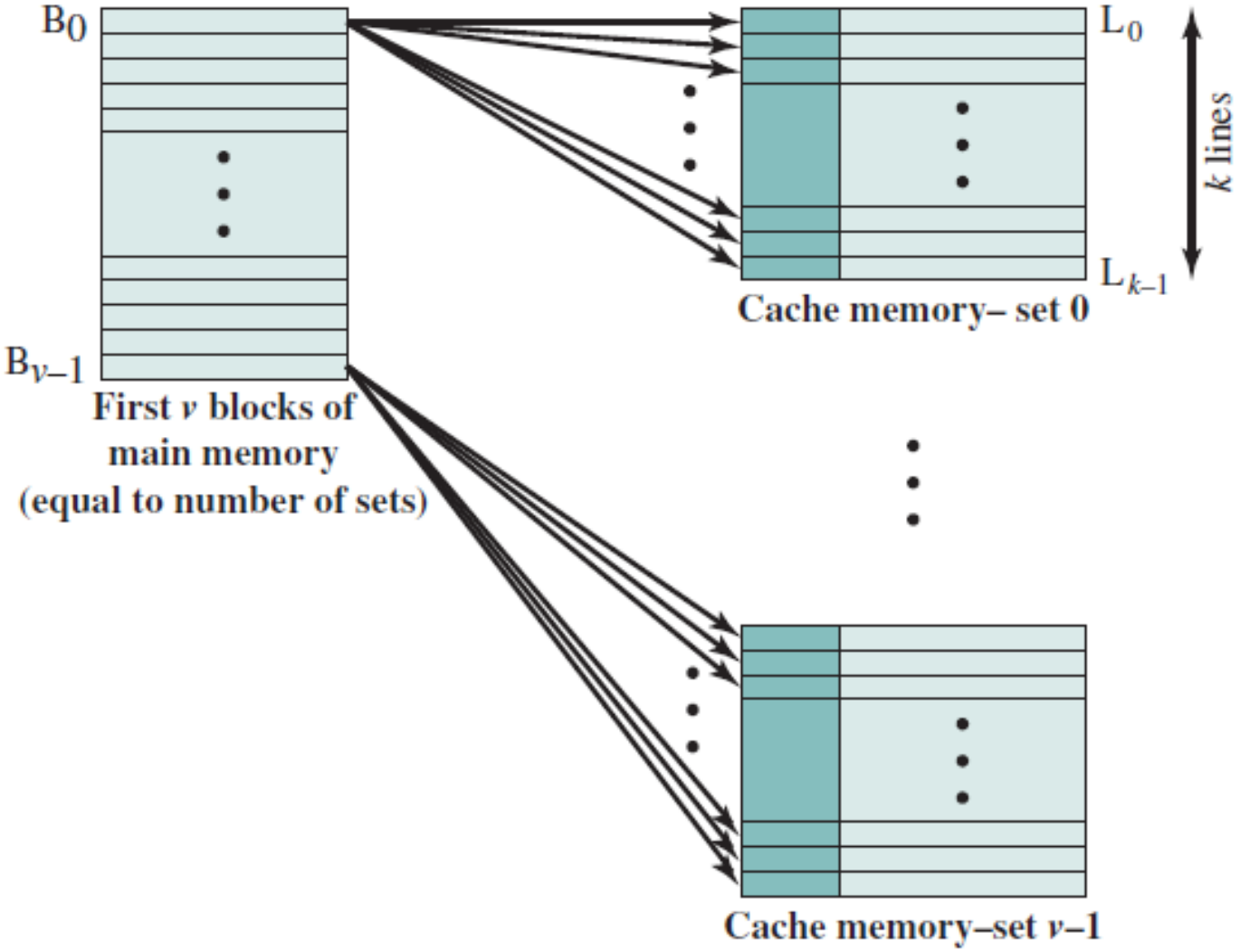
m = number of lines in the cache

v = number of sets

k = number of lines in each set

  - This is a **k-way set associative mapping**
  - A block B_j can be mapped to any of the blocks in set j

Cache – Set Associative Mapping



Cache – Set Associative Mapping

- Cache control logic interprets a memory address as three fields: Tag, Set, and Word
  - *d* Set bits specify one of *n* = *2^d* sets.
  - *s* bits of the Tag and Set fields specify one of the *2^s* blocks of main memory
- Tag in a memory address is much smaller and is only compared to the *k* tags within a single set

Address length $= (s + w)$ bits

Number of addressable units $= 2^{s+w}$ words or bytes

Block size = line size $= 2^w$ words or bytes

Number of blocks in main memory $= \dfrac{2^{s+w}}{2^w} = 2^s$

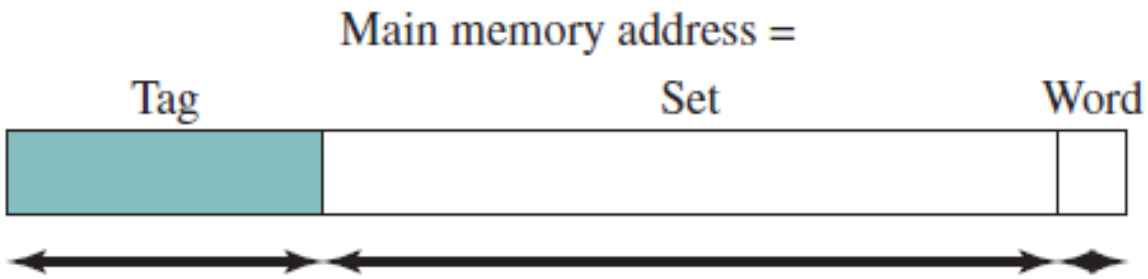
Number of lines in set $= k$

Number of sets $= v = 2^d$

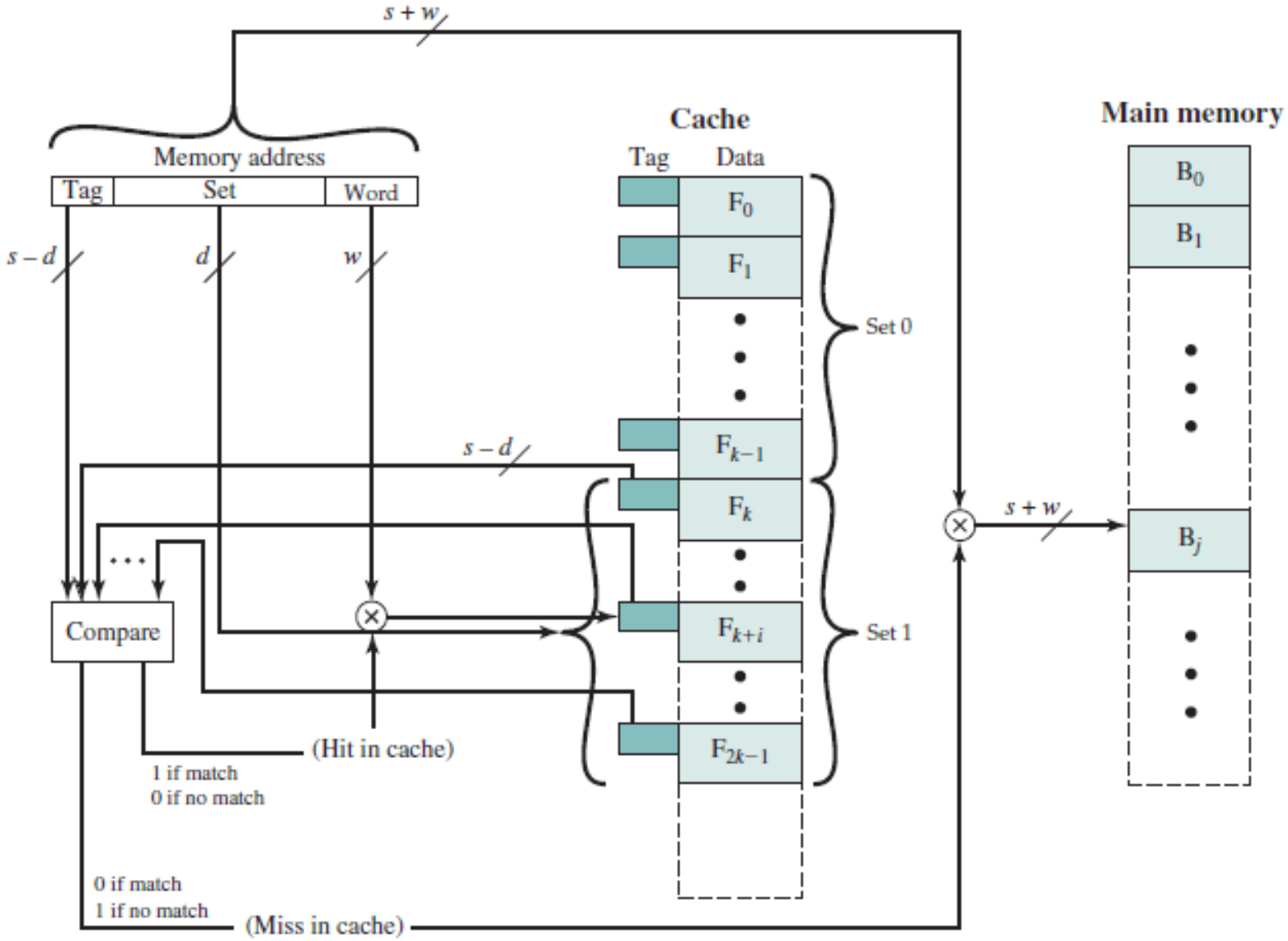
Number of lines in cache $= m = kv = k \times 2^d$

Size of cache $= k \times 2^{d+w}$ words or bytes

Size of tag $= (s - d)$ bits



Cache – Set Associative Mapping



Replacement Algorithms

- Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced
  - For direct mapping, there is only one possible line for any particular block – no choice is possible
  - For the associative and set-associative mapping, a replacement algorithm is needed
- Some common replacement algorithms are:
  - LRU
  - FIFO
  - LFU
  - Random

Replacement Algorithm – LRU

- Least Recently Used Algorithm
  - Replace that block in the set **that has been in the cache longest with no reference to it**
- To implement:
  - Cache maintains a separate list of indexes of all lines in cache
  - When a line is referenced, the line index moves to the front of the list
  - For replacement, the line at the back of the list is used
- Simple, popular and effective

Replacement Algorithm –FIFO, LFU ..

- First In First Out Algorithm
  - Replace that block in the set that has been in the cache longest
  - In other words, the block that came in first also goes out first
- Least Frequently Used Algorithm
  - Replace that block in the set that has experienced the fewest references
  - Implemented by associating a counter with each line; replace whichever line has the lowest value
- Random
  - Not a usage based algorithm
  - Pick any line at random and choose it for replacement
  - Only slightly inferior performance to algorithms based on usage

Exercise

- Given a cache of size 8KB capable of holding 4B blocks in a single line. For the hexadecimal main memory addresses 111111, 666666, BBBBBB, show the following information, in hexadecimal format:

  a. Tag, Line, and Word values for a direct-mapped cache.

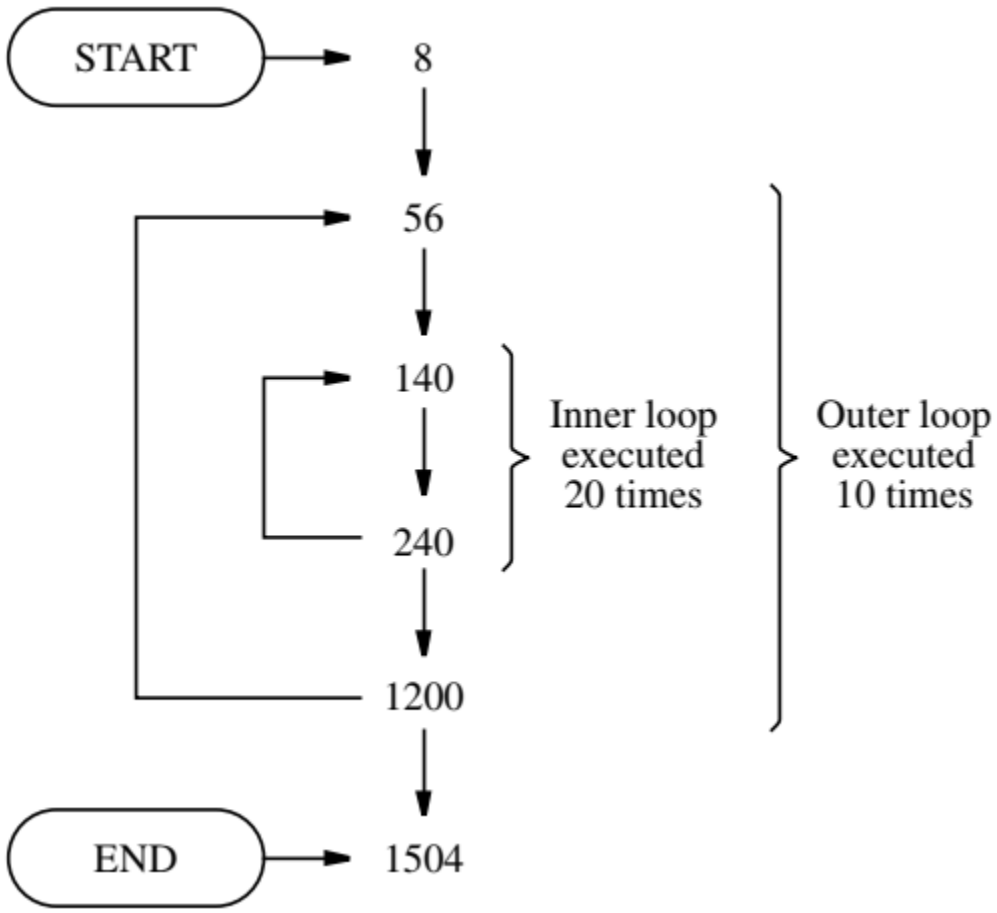
  b. Tag and Word values for an associative cache.

  c. Tag, Set, and Word values for a four-way set-associative cache.

  What is the size of the main memory?

Exercise

- The access times of the main memory and the Cache memory, in a computer system, are 500 n sec and 50 n sec, respectively. The hit ratio for the read access only is 0.9 and a write-through policy (where both main and cache memories are updated simultaneously) is used. Determine the average time of the main memory.
  - Consider, $t_{ave} = h * T_c + (1 - h) * T_m$ where T_c is the cache access time, T_m is the memory access time and h is the hit ratio
- A computer system has L1 and L2 cache with access times of 5ns and 10ns respectively. The main memory access time for a cache line is 200ns. During an execution a computer program experiences hit ratios of 0.95 in L1 and 0.99 in L2. Compute the average memory access time during the execution.
  - Consider, $t_{ave} = h_1 * T_{c_1} + (1 - h_1) * h_2 * T_{c_2} + (1 - h_1)(1 - h_2)T_m$ where T_{c_1} and T_{c_2} are the cache access time, T_m is the memory access time and h_1 and h_2 are the hit ratio

Problems

- The memory of a computer is byte-addressable, and the word length is 32 bits. A program consists of two nested loops—a small inner loop and a much larger outer loop. The general structure of the program is given in Figure P8.1. The decimal memory addresses shown delineate the location of the two loops and the beginning and end of the total program. All memory locations in the various sections of the program, 8-52, 56-136, 140-240, and so on, contain instructions to be executed in straight-line sequencing. The program is to be run on a computer that has an instruction cache organized in the direct-mapped manner (see Figure 8.16) with the following parameters: Cache size: 1KB, Block size 128B. The miss penalty in the instruction cache is 80τ, where τ is the access time of the cache. Compute the total time needed for instruction fetching during execution of the program in Figure P8.1.